

```
template<typename T>
class SegmentTree
{
private:
    struct node
    {
        T sum, maxk, mink, add, mul, setk;
        bool setked;
        node() : sum(0), maxk(numeric_limits<T>::max()),
mink(numeric_limits<T>::max()), add(0), mul(1), setk(0), setked(false) {}
    };
    int n;
    bool _Mod;
    T modk;
    vector<node> tr;
    T tr_mod(T val) const
    {
        if(_Mod)
        {
            return ((val % modk) + modk) % modk;
        }
        return val;
    }
    void push_up(int rt)
    {
        node &ls = tr[rt << 1], &rs = tr[rt << 1 | 1];
        tr[rt].sum = ls.sum + rs.sum;
        tr[rt].maxk = max(ls.maxk, rs.maxk);
        tr[rt].mink = min(ls.mink, rs.mink);
        if(_Mod)
        {
            tr[rt].sum = tr_mod(tr[rt].sum);
        }
    }
    void _range_set(int rt, int len, T val)
    {
        val = tr_mod(val);
        tr[rt].sum = tr_mod(val * len);
        tr[rt].maxk = val;
        tr[rt].mink = val;
        tr[rt].setk = val;
        tr[rt].setked = true;
        tr[rt].add = 0;
        tr[rt].mul = 1;
    }
    void _range_mul(int rt, T val)
    {
        val = tr_mod(val);
        tr[rt].sum = tr_mod(tr[rt].sum * val);
        tr[rt].maxk = tr_mod(tr[rt].maxk * val);
        tr[rt].mink = tr_mod(tr[rt].mink * val);
        tr[rt].mul = tr_mod(tr[rt].mul * val);
    }
};
```

```
        tr[rt].add = tr_mod(tr[rt].add * val);
    }
    void _range_add(int rt, int len, T val)
    {
        val = tr_mod(val);
        tr[rt].sum = tr_mod(tr[rt].sum + val * len);
        tr[rt].maxk = tr_mod(tr[rt].maxk + val);
        tr[rt].mink = tr_mod(tr[rt].mink + val);
        tr[rt].add = tr_mod(tr[rt].add + val);
    }
    void push_down(int rt, int pl, int pr)
    {
        if (pl == pr) return;
        int mid = (pl + pr) >> 1;
        int ls = rt << 1, rs = rt << 1 | 1;
        node &cur = tr[rt];
        if (cur.setked)
        {
            _range_set(ls, mid - pl + 1, cur.setk);
            _range_set(rs, pr - mid, cur.setk);
        }
        if (cur.mul != 1)
        {
            _range_mul(ls, cur.mul);
            _range_mul(rs, cur.mul);
            cur.mul = 1;
        }
        if (cur.add != 0)
        {
            _range_add(ls, mid - pl + 1, cur.add);
            _range_add(rs, pr - mid, cur.add);
            cur.add = 0;
        }
    }
    void build(int pl, int pr, int rt, const vector<T>& data)
    {
        if (pl == pr)
        {
            tr[rt].sum = tr_mod(data[pl]);
            tr[rt].maxk = tr_mod(data[pl]);
            tr[rt].mink = tr_mod(data[pl]);
            return;
        }
        int mid = (pl + pr) >> 1;
        build(pl, mid, rt << 1, data);
        build(mid + 1, pr, rt << 1 | 1, data);
        push_up(rt);
    }
    void update(int L, int R, T mul, T add, T setk, int pl, int pr, int rt)
    {
        if (L > pr || R < pl)
        {
            return;
        }
    }
```

```

        if (L <= pl && pr <= R)
        {
            if (setk != 0)
            {
                _range_set(rt, pr - pl + 1, setk);
            }
            else if (mul != 1)
            {
                _range_mul(rt, mul);
                if (add != 0)
                {
                    _range_add(rt, pr - pl + 1, add);
                }
            }
            else if (add != 0)
            {
                _range_add(rt, pr - pl + 1, add);
            }
            return;
        }
        push_down(rt, pl, pr);
        int mid = (pl + pr) >> 1;
        update(L, R, mul, add, setk, pl, mid, rt << 1);
        update(L, R, mul, add, setk, mid + 1, pr, rt << 1 | 1);
        push_up(rt);
    }
    T _query_sum(int L, int R, int pl, int pr, int rt)
    {
        if (L > pr || R < pl) return 0;
        if (L <= pl && pr <= R) return tr[rt].sum;
        push_down(rt, pl, pr);
        int mid = (pl + pr) >> 1;
        return tr_mod(_query_sum(L, R, pl, mid, rt << 1) + _query_sum(L, R,
mid + 1, pr, rt << 1 | 1));
    }
    T _query_max(int L, int R, int pl, int pr, int rt)
    {
        if (L > pr || R < pl) return numeric_limits<T>::min();
        if (L <= pl && pr <= R)
        {
            return tr[rt].maxk;
        }
        push_down(rt, pl, pr);
        int mid = (pl + pr) >> 1;
        return max(_query_max(L, R, pl, mid, rt << 1), _query_max(L, R, mid
+ 1, pr, rt << 1 | 1));
    }
    T _query_min(int L, int R, int pl, int pr, int rt)
    {
        if (L > pr || R < pl) return numeric_limits<T>::max();
        if (L <= pl && pr <= R)
        {
            return tr[rt].mink;
        }
    }

```

```

        push_down(rt, pl, pr);
        int mid = (pl + pr) >> 1;
        return min(_query_min(L, R, pl, mid, rt << 1), _query_min(L, R, mid
+ 1, pr, rt << 1 | 1));
    }
    template<typename Compare>
    int find_bound(int L, int R, int pl, int pr, int rt, T val, Compare
comp)
    {
        if (pr < L || pl > R)
        {
            return -1;
        }
        if (pl == pr)
        {
            return (pl >= L && pr <= R && comp(tr[rt].mink, val)) ? pl :
-1;
        }
        push_down(rt, pl, pr);
        int mid = (pl + pr) >> 1;
        int left = find_bound(L, R, pl, mid, rt << 1, val, comp);
        if (left != -1)
        {
            return left;
        }
        return find_bound(L, R, mid + 1, pr, rt << 1 | 1, val, comp);
    }
    public:
    SegmentTree(const vector<T>& data, T mod = 0) : n(data.size()),
modk(mod), _Mod(mod > 0)
    {
        if (n == 0) throw invalid_argument("data size is empty");
        tr.resize(4 * n);
        build(0, n - 1, 1, data);
    }
    void range_set(int L, int R, T val)
    {
        update(L, R, 1, 0, val, 0, n - 1, 1);
    }
    void range_add(int L, int R, T val)
    {
        update(L, R, 1, val, 0, 0, n - 1, 1);
    }
    void range_mul(int L, int R, T val)
    {
        update(L, R, val, 0, 0, 0, n - 1, 1);
    }
    T query_sum(int L, int R)
    {
        return _query_sum(L, R, 0, n - 1, 1);
    }
    T query_max(int L, int R)
    {
        return _query_max(L, R, 0, n - 1, 1);
    }

```

```

}
T query_min(int L, int R)
{
    return _query_min(L, R, 0, n - 1, 1);
}
int lower_bound(int L, int R, T val) // 无法找到时会返回 -1, 而不是最后位置
的下一个位置
{
    return find_bound(L, R, 0, n - 1, 1, val, [](T x, T y){return x >=
y;});
}
int upper_bound(int L, int R, T val)
{
    return find_bound(L, R, 0, n - 1, 1, val, [](T x, T y){return x >
y;});
}
};

const double pi=acos(-1.0);
const double eps=1e-6;
int sgn(double x){ //判断x得大小
    if(fabs(x)<eps)return 0;
    return x<0?-1:1;
}
struct Point{ //点
    double x,y;
    Point(){}
    Point(double x,double y):x(x),y(y){}
    Point operator+(Point B){
        return Point(x+B.x,y+B.y);
    }
    Point operator-(Point B){
        return Point(x-B.x,y-B.y);
    }
    Point operator*(double k){
        return Point(x*k,y*k);
    }
    Point operator/(double k){
        return Point(x/k,y/k);
    }
    bool operator==(Point B){
        return sgn(x-B.x)==0&&sgn(y-B.y)==0;
    }
    bool operator<(Point B){ //先x升序, 然后y升序
        return sgn(x-B.x)<0|| (sgn(x-B.x)==0&&sgn(y-B.y)<0);
    }
};
double Distance(Point A,Point B){ //两点的距离

```

```

    return sqrt((A.x-B.x)*(A.x-B.x)+(A.y-B.y)*(A.y-B.y));
}
typedef Point Vector;                                //平移到原点，则可看成
从(0,0)到(x,y)
double Dot(Vector A,Vector B){                        //点积
A*B==|A|*|B|*cosx==A.x*B.x+A.y*B.y 可用于判断角大小
    return A.x*B.x+A.y*B.y;
}
double Len(Vector A){                                //向量的长度
    return sqrt(Dot(A,A));
}
double Len2(Vector A){                                //向量的平方，避免开方
    return Dot(A,A);
}
double Angle(Vector A,Vector B){                     //A与B的夹角
    return acos(Dot(A,B)/Len(A)/Len(B));
}
double Cross(Vector A,Vector B){                     //叉积
A*B==|A|*|B|*sinx 面积有正负 x为从A逆时针转到B的角度
    return A.x*B.y-A.y*B.x;                          //>0:B在A的逆时针方
向. <0:B在A的顺时针方向 =0:共线
}
Vector Rotate(Vector A,double rad){                  //向量逆时针旋转rad度
    return Vector(A.x*cos(rad)-A.y*sin(rad),A.x*sin(rad)+A.y*cos(rad));
}
Vector Normal(Vector A){                              //法向量
    return Vector(-A.y/Len(A),A.x/Len(A));
}
bool Parallel(Vector A,Vector B){                    //是否平行或重合
    return sgn(Cross(A,B))==0;
}
struct Line{
    Point p1,p2;
    Line(){ }
    Line(Point p1,Point p2):p1(p1),p2(p2){ }
    Line(Point p,double angle){                       //一个点一个斜率
        p1=p;
        if(sgn(angle-pi/2)==0){
            p2=(p1+Point(0,1));
        }
        else{
            p2=(p1+Point(1,tan(angle)));
        }
    }
    Line(double a,double b,double c){                 //ax+by+c=0
        if(sgn(a)==0){
            p1=Point(0,-c/b);
            p2=Point(1,-c/b);
        }
        else if(sgn(b)==0){
            p1=Point(-c/a,0);
            p2=Point(-c/a,1);
        }
        else{

```

```

        p1=Point(0,-c/b);
        p2=Point(1,(-c-a)/b);
    }
}
};

int Point_line_relation(Point p,Line v){           //点和线的关系
    int c=sgn(Cross(p-v.p1,v.p2-v.p1));
    if(c<0)return 1;                               //p在v的左侧
    if(c>0)return 2;                               //p在v的右侧
    return 0;                                       //p在v上
}

bool Point_on_seg(Point p,Line v){                //点是否在线段上, 0:
点p不在线段上, 1: 点p在线段上
    return sgn(Cross(p-v.p1,v.p2-v.p1))==0&&sgn(Dot(p-v.p1,p-v.p2))<=0;
}

double Dis_point_line(Point p,Line v){            //点到直线的距离
    return abs(Cross(p-v.p1,v.p2-v.p1))/Distance(v.p1,v.p2);
}

Point Point_line_proj(Point p,Line v){            //点在直线上的投影点
    double k=Dot(v.p2-v.p1,p-v.p1)/Len2(v.p2-v.p1);
    return v.p1+(v.p2-v.p1)*k;
}

Point Point_line_symmetry(Point p,Line v){        //点关于直线的对称点
    Point q=Point_line_proj(p,v);
    return Point(2*q.x-p.x,2*q.y-p.y);
}

typedef Line Segment;                             //线段
double Dis_point_seg(Point p,Segment v){          //点到线段的距离
    if(sgn(Dot(p-v.p1,v.p2-v.p1))<0||sgn(Dot(p-v.p2,v.p1-v.p2))<0){
        return min(Distance(p,v.p1),Distance(p,v.p2));
    }
    return Dis_point_line(p,v);
}

int Line_relation(Line v1,Line v2){              //两条直线的位置关系
    if(sgn(Cross(v1.p2-v1.p1,v2.p2-v2.p1))==0){   //
        if(Point_line_relation(v1.p1,v2)==0)return 1; //1: 重合
        else return 0;                             //0: 平行
    }
    return 2;                                       //2: 相交
}

Point Cross_point(Point a,Point b,Point c,Point d){ //两条直线的交点, 应保
证s2-s1!=0,即直线AB与CD不共线不平行
    double s1=Cross(b-a,c-a);
    double s2=Cross(b-a,d-a);                     //叉积有正负
    return Point(c.x*s2-d.x*s1,c.y*s2-d.y*s1);
}

bool Cross_segment(Point a,Point b,Point c,Point d){ //线段AB与线段CD是否
相交
    double c1=Cross(b-a,c-a),c2=Cross(b-a,d-a);
    double d1=Cross(d-c,a-c),d2=Cross(d-c,b-c);
    return sgn(c1)*sgn(c2)<0&&sgn(d1)*sgn(d2)<0;   //1: 相交, 0: 不相交
}

int Point_in_polygon(Point pt,Point *p,int n){    //判断pt这个点在不
在(*p)这个多边形内部

```

```

    for(int i=0;i<n;i++){
        if(p[i]==pt)return 3; //3: 点在多边形的顶点
    }
    for(int i=0;i<n;i++){
        Line v=Line(p[i],p[(i+1)%n]);
        if(Point_on_seg(pt,v))return 2; //2: 点在多边形的边上
    }
    int num=0;
    for(int i=0;i<n;i++){
        int j=(i+1)%n;
        int c=sgn(Cross(pt-p[j],p[i]-p[j]));
        int u=sgn(p[i].y-pt.y);
        int v=sgn(p[j].y-pt.y);
        if(c>0&&u<0&&v>=0)num--;
        if(c<0&&u>=0&&v<0)num--;
    }
    return num!=0; //1: 点在内部; 0: 点在
外部
}
double Polygon_area(Point *p,int n){ //求多边形的面积
    double area=0;
    for(int i=0;i<n;i++){
        area+=Cross(p[i],p[(i+1)%n]);
    }
    return area/2; //面积有正负, 不能简单
的取绝对值
}
Point Polygon_center(Point *p,int n){ //求多边形的重心
    Point ans(0,0);
    if(Polygon_area(p,n)==0)return ans;
    for(int i=0;i<n;i++){
        ans=ans+(p[i]+p[(i+1)%n])*Cross(p[i],p[(i+1)%n]);
    }
    return ans/Polygon_area(p,n)/6; //这里就可能需要负的面
积
}
int Convex_hull(Point *p,int n,Point *ch){ //点集 p 求凸包,放入
ch点集中, 返回长度
    n=unique(p,p+n)-p;
    sort(p,p+n);
    int v=0;
    for(int i=0;i<n;i++){
        while(v>1&&sgn(Cross(ch[v-1]-ch[v-2],p[i]-ch[v-1]))<=0){
            v--;
        }
        ch[v++]=p[i];
    }
    int j=v;
    for(int i=n-2;i>=0;i--){
        while(v>j&&sgn(Cross(ch[v-1]-ch[v-2],p[i]-ch[v-1]))<=0){
            v--;
        }
        ch[v++]=p[i];
    }

```



```

    }
    if(n>1)v--;
    return v;
}
struct Circle{ //圆
    Point c;
    double r;
    Circle(){}
    Circle(Point c,double r):c(c),r(r){}
    Circle(double x,double y,double _r){
        c=Point(x,y);
        r=_r;
    }
};

int Line_circle_relation(Line v,Circle C){ //直线和圆的关系
    double dst=Dis_point_line(C.c,v);
    if(sgn(dst-C.r)<0)return 0; //0: 直线和圆相交
    if(sgn(dst-C.r)==0)return 1; //1: 线段和圆相切
    return 2; //2: 直线在圆外
}

int Seg_circle_relation(Segment v,Circle C){ //线段和圆的关系
    double dst=Dis_point_seg(C.c,v);
    if(sgn(dst-C.r)<0)return 0; //0: 线段在圆内
    if(sgn(dst-C.r)==0)return 1; //1: 线段与圆相切
    return 2; //2: 线段在圆外
}

}

template <class T>
struct Fenwick
{
    int n;
    vector<T> t;
    Fenwick(T n) : n(n) { t.assign(n + 1, T{}); }
    void add(int x, const T &v)
    {
        for (int i = x; i <= n; i += i & -i)
        {
            t[i] += v;
        }
    }
    T sum(int x)
    {
        assert(x >= 0);
        int res = 0;
        for (int i = x; i; i -= i & -i)
        {
            res += t[i];
        }
        return res;
    }
    T range(int l, int r)

```

```
{
    return sum(r) - sum(l - 1);
}
int select(const T &k)
{
    int x = 0;
    T cur{};
    for (int i = 1 << __lg(n); i; i /= 2)
    {
        if (x + i <= n && cur + t[x + i] <= k)
        {
            x += i;
            cur += t[x];
        }
    }
    return x;
}
};
```

```
inline ll read(){
    char ch=getchar();
    register ll s=0,w=1;
    while(ch<'0' || ch>'9'){
        if(ch=='-')w=-1;
        ch=getchar();
    }
    while(ch>='0' && ch<='9'){
        s=(s<<1)+(s<<3)+(ch^48);
        ch=getchar();
    }
    return s*w;
}
inline void print(ll x){
    if(x<0){
        putchar('-');
        x=-x;
    }
    if(x>9){
        print(x/10);
    }
    putchar(x%10+'0');
}
```

```
ll fast_pow(ll x,ll y,int m){
    ll res=1;
    x%=m;
    while(y){
        if(y&1) res=res*x%m;
        x=x*x%m;
        y>>=1;
    }
```

```
    }
    return res;
}

bool witness(ll a, ll n) { // 是合数则返回true, 素数则返回false
    ll u = n - 1;
    int t = 0;
    while(u & 1 == 0) u = u >> 1, t++;
    ll x1, x2;
    x1 = fast_pow(a, u, n);
    for(int i = 1; i <= t; i++) {
        x2 = fast_pow(x1, 2, n);
        if(x2 == 1 && x1 != -1 && x1 != n - 1) {
            return true;
        }
        x1 = x2;
    }
    if(x1 != 1) return true;
    return false;
}

bool miller_rabin(ll n, int s) { // 对 n 进行 s 次测试
    if(n < 2) return false;
    if(n == 2) return true;
    if(n % 2 == 0) return false;
    for(int i = 0; i < s && i < n; i++) {
        ll a = rand() % (n - 1) + 1;
        if(witness(a, n)) return false;
    }
    return true;
}
```